

Multi-Agent Reinforcement Learning With Spatial–Temporal Attention for Flocking With Collision Avoidance of a Scalable Fixed-Wing UAV Fleet

Chao Yan^{ID}, *Member, IEEE*, Chang Wang^{ID}, Han Zhou, Xiaojia Xiang^{ID},
Xiangke Wang^{ID}, *Senior Member, IEEE*, and Lincheng Shen

Abstract—Flocking with multiple unmanned aerial vehicles (UAVs) offers significant potential for diverse applications due to its enhanced maneuverability, improved efficiency, and increased robustness. Collision avoidance is a critical and challenging issue for distributed flocking control with a UAV fleet, especially in dynamic environments with varying numbers of non-cooperative intruders. However, existing reinforcement learning based methods mainly focus on flocking with collision avoidance tasks with static obstacles and a fixed number of UAVs. In this article, we propose a scalable multi-agent reinforcement learning based method to solve the distributed flocking with collision avoidance problem for a scalable fleet of fixed-wing UAVs in dynamic environments. Specifically, we cast this problem in a decentralized partially observable Markov decision process framework and propose a scalable multi-agent reinforcement learning algorithm called spatial-temporal attention multi-agent actor-critic (STAAC). In this algorithm, we design a spatial-temporal attention based population-invariant network architecture to facilitate the representation learning of dynamic dimensional observations. By integrating the local spatial attention and global temporal attention mechanisms, STAAC is able to adapt to the changes in the scale of UAV fleets and the number of intruders. Finally, we empirically demonstrate the effectiveness, scalability, and adaptability of the proposed approach in numerical simulations and hardware-in-the-loop experiments.

Index Terms—Fixed-wing UAV, flocking, collision avoidance, multi-agent reinforcement learning, spatial-temporal attention.

I. INTRODUCTION

OVER the past two decades, unmanned aerial vehicles (UAVs) have emerged as invaluable platforms in various industries and applications, owing to their distinctive

Received 23 November 2023; revised 4 May 2024, 18 July 2024, and 27 September 2024; accepted 11 November 2024. Date of publication 5 December 2024; date of current version 4 February 2025. This work was supported in part by the Natural Science Foundation of Jiangsu Province under Grant BK20241396 and in part by the National Natural Science Foundation of China under Grant 62403240. The Associate Editor for this article was L. Wang. (*Corresponding author: Xiaojia Xiang.*)

Chao Yan is with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: yanchao@nuaa.edu.cn).

Chang Wang, Han Zhou, Xiaojia Xiang, Xiangke Wang, and Lincheng Shen are with the College of Intelligence Science and Technology, National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: wangchang07@nudt.edu.cn; zhouhan@nudt.edu.cn; xiangxiaojia@nudt.edu.cn; xkwang@nudt.edu.cn; lcsen@nudt.edu.cn).

Digital Object Identifier 10.1109/TITS.2024.3505929

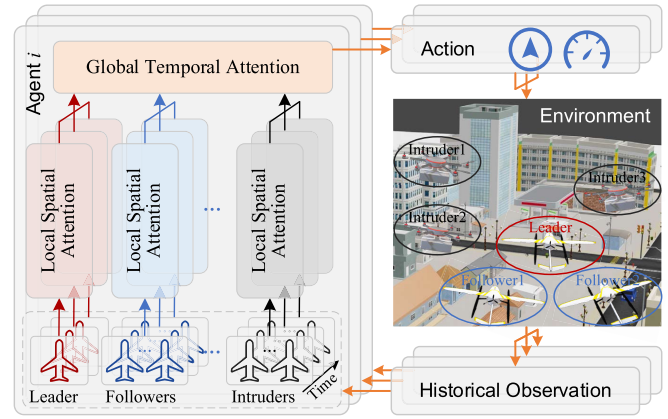


Fig. 1. Illustration of the proposed spatial-temporal attention multi-agent actor-critic (STAAC) approach for distributed flocking with collision avoidance of fixed-wing UAV fleets in dynamic environments.

advantages in flexibility and versatility [1]. A swarm of UAVs can perform specific tasks that are beyond the capabilities of an individual UAV, offering significant potential for diverse applications such as area coverage [2], emergency rescue [3], and delivery logistics [4]. As a result, UAV swarms have evolved into an attractive field of research. One open question is how to coordinate multiple UAVs to accomplish complicated tasks in a distributed manner.

Emulating collective behaviors in nature potentially provides a promising solution for this problem [5]. Flocking is a pervasive form of collective behavior that can be observed in various natural systems [6]. Inspired by flocking behaviors, flocking with multiple UAVs has attracted considerable attention due to its characteristics in agility, robustness, and redundancy [7], [8]. The complexity of coordinating a flock of UAVs to accomplish tasks together increases dramatically with the number of UAVs and the dynamics of environments. Despite significant efforts have been devoted aggressively, developing a distributed flocking controller for multiple UAVs is still a huge challenge, especially in dynamic environments with scalable fleets of UAVs and varying numbers of non-cooperative intruders.

In recent years, deep reinforcement learning (RL) has emerged as a powerful technique for solving complex decision-making tasks in various domains [9]. Due to its advantages in model freedom, flexibility, and computational cost, a growing number of researchers believe that developing deep RL techniques in the field of swarm robotics is promising. Consequently, deep RL has become increasingly popular for addressing the UAV flocking control problem. Wang et al. [10] proposed an oracle-guided two-stage learning framework consisting of deep RL and supervised learning to learn a flocking and navigation policy for a large-scale UAV swarm. Xiao et al. [11] proposed a homogeneous graph neural network based multi-agent RL algorithm to solve the leader-follower flocking problem in stochastic environments.

The above studies focus only on rotary UAVs and are not applicable to fixed-wing UAVs. Compared to rotary UAVs, fixed-wing UAVs have advantages in terms of range, endurance, and flight speed [12]. Consequently, flocking with fixed-wing UAVs has more potential applications, especially in industrial and military activities [13]. However, due to the complex dynamics and nonholonomic constraints of fixed-wing UAVs [14], it is more difficult to achieve flocking with a fleet of fixed-wing UAVs. To date, the flocking problem of fixed-wing UAVs has not been well studied.

The effectiveness of deep RL for flocking with fixed-wing UAVs has been preliminarily demonstrated. Hung and Givigi [14] first formulated the leader-follower flocking problem for fixed-wing UAVs in the RL framework. However, they only considered the interaction between the leader and each individual follower, ignoring the interactions among all the followers. In our recent work [15], [16], we investigated the flocking with collision avoidance problem for a fleet of fixed-wing UAVs using multi-agent RL. We alleviated the scalability issue through contrasting local situation maps [15] or customizing a specialized network structure [16]. Nevertheless, the previous work [15], [16] assumed that the fixed-wing UAVs flew in free space, without considering other entities or obstacles in the environment for simplicity.

Collision avoidance is a fundamental and critical issue for UAV maneuvering in complex environments [17]. Many researchers have successfully applied deep RL to solve this problem. Ou et al. [18] proposed a two-stage learning scheme combining unsupervised learning and deep RL to solve the obstacle avoidance problem for UAVs with monocular vision. Wang et al. [19] designed a deep RL based obstacle avoidance controller for UAV navigation with a non-expert prior whose input contained only partial environmental information measured by distance sensors. However, these methods were only validated with a single rotary UAV.

In a scenario involving multiple UAVs and obstacles, the collision avoidance problem becomes more challenging because UAV-obstacle collisions and UAV-UAV collisions must be considered simultaneously. Shen et al. [20] combined deep RL and digital twin technologies to address the flocking motion problem for multiple UAVs in complex environments. Bai et al. [21] integrated the repulsion function into deep RL, endowing the UAV swarms with the capabilities of collision avoidance and flock topology maintenance. The existing stud-

ies mainly focus only on rotary UAVs, while the multi-agent RL-based obstacle avoidance methods for multiple fixed-wing UAVs are rather scarce. The most related research is our recent work [22], where a multi-stage multi-agent RL algorithm for flocking with multiple fixed-wing UAVs in obstacle-cluttered environments was proposed. However, the existing methods, whether for rotary or fixed-wing UAVs, only considered static obstacles in the flocking scenario. The effectiveness of these methods in more complex scenarios with dynamic obstacles needs to be further investigated.

In this article, we propose a novel spatial-temporal attention multi-agent actor-critic (STAAC) algorithm to learn the distributed flocking with collision avoidance policy for a scalable fleet of fixed-wing UAVs in dynamic environments. Compared with our previous work [23], in which we proposed an attention-based population-invariant multi-agent RL approach for scalable fixed-wing UAV swarms, this article presents significant extensions in the following aspects: i) Yan et al. [23] only addressed the collision-free flocking problem of a scalable fixed-wing UAV swarm in free space. In contrast, we consider a more challenging environment that includes not only scalable swarms of fixed-wing UAVs but also a variable number of non-cooperative intruders. ii) Yan et al. [23] only used the self-attention mechanism with the multi-agent deep deterministic policy gradient algorithm (MADDPG) algorithm to learn the flocking policy. Nevertheless, we propose a scalable and efficient learning algorithm to tackle the more challenging task involving UAV-UAV collision avoidance and UAV-intruder collision avoidance. As illustrated in Figure 1, we first divide the entities within the observable range of each UAV into distinct groups according to their characteristics and properties. We then design a population-invariant network architecture for representation learning of dynamic dimensional observations by integrating local spatial attention and global temporal attention mechanisms. Furthermore, we improve the MADDPG algorithm [24] using parameter sharing [25] and clipped double Q-learning [26] techniques to efficiently optimize the scalable control policy. Through a series of numerical simulations and hardware-in-the-loop (HITL) experiments, we demonstrate the significant superiority of the proposed STAAC approach in terms of policy optimisation and scalability.

To the best of our knowledge, we are the first to solve the distributed flocking with collision avoidance problem for a scalable fixed-wing UAV fleet in dynamic environments using multi-agent RL. The main contributions of this article are summarized as follows:

- 1) We design a population-invariant network architecture integrating the local spatial attention and global temporal attention mechanisms to represent dynamic dimensional observations. With this architecture, the learned policy can adapt to different scenarios with varying numbers of followers and intruders in a zero-shot manner.
- 2) We propose an efficient multi-agent actor-critic learning algorithm based on the parameter sharing and clipped double Q-learning techniques to optimize the scalable flocking with collision avoidance policy. This algorithm

enhances adaptability to changes in fleet size, improving learning efficiency and robustness.

- 3) We empirically validate the effectiveness, scalability, and generalizability of the proposed STAAC algorithm through numerical simulations and HITL experiments. The results demonstrate its superior performance in terms of flocking behavior, collision avoidance, and adaptability to diverse and practical scenarios.

The rest of this article is organized as follows. Section II formulates the flocking with collision avoidance problem, followed by the proposed STAAC algorithm in Section III. Section IV presents the simulation results. Finally, Section V concludes this article.

II. PROBLEM FORMULATION

In this section, we describe the distributed flocking and collision avoidance problem for a scalable fleet of fixed-wing UAVs and then formulate this problem as a decentralized partially observable Markov decision process (Dec-POMDP).

A. Problem Definition

In our flocking scenario, a scalable fleet of fixed-wing UAVs flies in a two-dimensional dynamic environment cluttered with a set of non-cooperative intruders (see Figure 1 (right)). Specifically, one unique leader is followed by a set of identical followers $\mathcal{N} := \{1, 2, \dots, n\}$. The UAV fleet should avoid potential collisions with the set of intruders $\mathcal{M} := \{1, 2, \dots, m\}$. Note that both the number of followers and the number of intruders (i.e., n and m) are not fixed. Our goal is to develop a distributed flocking control policy that enables varying numbers of followers to flock with the leader while avoiding collisions between the followers and the intruders by adjusting the heading and speed actions in real time. In contrast to previous work, this problem is more intractable due to the size of the UAV fleet and the number of intruders are dynamic. The flocking controller must be scalable and adapt to changes in the number of entities. This requires a population-invariant multi-agent RL algorithm, enabling the learned flocking controller to maintain scalability.

B. System Model

We first model the kinematics of the fixed-wing UAV and the intruder, then the sensor model of the UAV.

1) *Kinematic Model*: Considering a fleet of fixed-wing UAVs flying at a fixed altitude, we can use a 4-tuple $\xi := (x, y, \psi, v)$, i.e., planar position, heading angle, and forward speed, to represent the state of each UAV. Referring to [22] and [27], we establish the following kinematics to describe the movement of the fixed-wing UAV:

$$\dot{\xi} = \frac{d}{dt} \begin{bmatrix} x \\ y \\ \psi \\ v \end{bmatrix} = \begin{bmatrix} v \cos \psi + \eta_x \\ v \sin \psi + \eta_y \\ f_\psi(\psi, \psi_{\text{set}}) \\ f_v(v, v_{\text{set}}) \end{bmatrix}, \quad (1)$$

where η_x and η_y are the disturbance terms following normal distributions $\mathbb{N}(0, \sigma_x^2)$ and $\mathbb{N}(0, \sigma_y^2)$, respectively.

$f_\psi(\psi, \psi_{\text{set}})$ and $f_v(v, v_{\text{set}})$ denote the heading and speed dynamics, respectively, which will be explained in more detail in the next section.

For the heading dynamics, we use a second-order system similar to [14] and [28] to simulate the response relationship between the heading angle setpoint ψ_{set} and the actual heading angle ψ . To make the response more realistic, we assume the parameters of the second-order system, i.e., the undamped natural frequency ω_n and the damping ratio ζ , are drawn from normal distributions $\mathbb{N}(\bar{\omega}_n, \sigma_\omega^2)$ and $\mathbb{N}(\bar{\zeta}, \sigma_\zeta^2)$, respectively. For the speed dynamics, we simulate the response relationship between the forward speed setpoint v_{set} and the actual forward speed v with a simple linear model. To reduce the computational complexity, the average forward speed is allowed to replace the integral process when calculating the planar position of a UAV [28]. We also introduce stochasticity into the calculation process. The average forward speed $\bar{v}$ of the UAV in an interval is expressed as follows:

$$\bar{v} = \frac{1}{2}(v + v_{\text{set}}) + \epsilon_v, \quad \epsilon_v \sim \mathbb{N}(0, \sigma_v^2), \quad (2)$$

where $\mathbb{N}(0, \sigma_v^2)$ is a normal distribution with zero mean. Note that $\bar{v}$ holds constant until the next forward speed setpoint is updated.

For fixed-wing UAVs, the asymmetrical constraints of forward speed must be considered. In addition, both heading rate and forward acceleration are constrained by saturation. Formally, the constraints are formulated as follows [27] and [29]:

$$\begin{cases} 0 < v_{\min} \leq v \leq v_{\max} \\ -\omega_{\max} \leq \omega \leq \omega_{\max} \\ -u_{\max} \leq u \leq u_{\max}, \end{cases} \quad (3)$$

where $v_{\min}$ and $v_{\max}$ denotes the minimum and maximum forward speeds, respectively. $\omega := \dot{\psi}_{\text{set}} - \dot{\psi}$ is the heading rate of the UAV, whose maximum value is denoted by $\psi_{\max}$. $u := v_{\text{set}} - v$ is the forward acceleration, and the corresponding maximum value is denoted by $\omega_{\max}$.

The kinematics of intruders are the same as those of fixed-wing UAVs. The only difference is that the above constraints are not taken into account. It is noticed that the established kinematic model (1) is stochastic and uncertain. Therefore, it is difficult to design control laws for flocking control using traditional model-based methods. In contrast, we use the stochastic kinematic model to generate simulated samples, and then leverage model-free multi-agent RL methods to learn the flocking control with collision avoidance policy.

2) *Sensor Model*: The sensing capabilities of the followers are equal and limited. Each follower can only access the state information of nearby followers and intruders. Let R_c denote the maximum sensing range of the follower, then the set of *neighbor-followers* of each follower i at the time step t is

$$\mathcal{F}_i^t := \{j \mid D_{i,j}^t < R_c, j \in \mathcal{N}, j \neq i\}, \quad (4)$$

where $D_{i,j}^t$ denotes the distance between the i -th follower and the j -th follower at the time step t . Analogously, the set of *neighbor-intruders* of each follower i at the time step t is

$$\mathcal{I}_i^t := \{k \mid D_{i,k}^t < R_c, k \in \mathcal{M}\}, \quad (5)$$

where $D_{i,k}^t$ denotes the distance between the i -th follower and the k -th intruder at the time step t .

C. Dec-POMDP Model

We regard the followers as agents and model the flocking control problem as a Dec-POMDP. The observation, state, action, and reward of each follower are defined as follows.

1) *Observation Space*: Given the limited sensing capabilities of the followers, the flocking environment is partially observable. To mitigate this problem, we stack the last four frames of historical information for decision-making [30], [31]. The stacked observation of each follower $i \in \mathcal{N}$ at the time step t can be defined as follows:

$$o_i^t := \bigcup_{\tau=t}^{t-3} (\xi_\tau^{\text{lea}}, \xi_{i,\tau}^{\text{fol}}, \xi_{\mathcal{F}_i^\tau}^{\text{fol}}, \xi_{\mathcal{I}_i^\tau}^{\text{int}}), \quad (6)$$

where ξ_τ^{lea} is the leader's state at the time step τ ; $\xi_{i,\tau}^{\text{fol}}$ is the follower i 's state at the time step τ ; $\xi_{\mathcal{F}_i^\tau}^{\text{fol}} := \{\xi_{j,\tau}^{\text{fol}} \mid j \in \mathcal{F}_i^\tau\}$ is the set of states of follower i 's neighbor-followers; $\xi_{\mathcal{I}_i^\tau}^{\text{int}} := \{\xi_{k,\tau}^{\text{int}} \mid k \in \mathcal{I}_i^\tau\}$ is the set of states of follower i 's neighbor-intruders.

2) *State Representation*: The system state includes the states of all the entities. Similar to the observation (6), the system state can be represented as

$$s^t := \bigcup_{\tau=t}^{t-3} (\xi_\tau^{\text{lea}}, \xi_\tau^{\text{fol}}, \xi_\tau^{\text{int}}), \quad (7)$$

where $\xi_\tau^{\text{fol}} := \{\xi_{j,\tau}^{\text{fol}} \mid j \in \mathcal{N}\}$ is the set of states of the followers; $\xi_\tau^{\text{int}} := \{\xi_{k,\tau}^{\text{int}} \mid k \in \mathcal{M}\}$ is the set of states of the intruders.

3) *Action Space*: All the followers have the same action space. Considering the kinematic constraints (3), we define the action in continuous spaces for each follower i :

$$a_i^t := (\omega_i^t, u_i^t), \quad (8)$$

where $\omega_i^t \in [-\omega_{\max}, \omega_{\max}]$, $u_i^t \in [-u_{\max}, u_{\max}]$.

4) *Reward Function*: Recalling the objective of flocking control, i.e., avoiding collisions with the followers and intruders while flocking with the leader, the reward function for each follower i is defined as:

$$r_i^t = r_{i,t}^{\text{lea}} + \sum_{j=1}^{|\mathcal{F}_i^t|} r_{i,j,t}^{\text{fol}} + \sum_{k=1}^{|\mathcal{I}_i^t|} r_{i,k,t}^{\text{int}}, \quad (9)$$

where $r_{i,t}^{\text{lea}}$, $r_{i,j,t}^{\text{fol}}$, and $r_{i,k,t}^{\text{int}}$ denote the *leader following reward*, *UAV-UAV collision avoidance reward*, and *UAV-intruder collision avoidance reward*, respectively. Specifically, to encourage the follower i to flock with the leader, $r_{i,t}^{\text{lea}}$ is defined as

$$r_{i,t}^{\text{lea}} = \begin{cases} -P_1 & \rho_i^t \leq R_s \\ w_1(\rho_i^t - R_a) & R_s < \rho_i^t \leq R_a \\ -(\rho_i^t - R_a) & \rho_i^t > R_a, \end{cases} \quad (10)$$

where ρ_i^t is the distance from the leader to the follower i at time step t ; $P_1 \gg 0$ is a large positive constant; $w_1 > 0$ is a

tuning parameter; R_a and R_s are the alert radius and minimum safety radius of the follower, respectively.

To punish the follower i for colliding with the j -th neighbor-followers, $r_{i,j,t}^{\text{fol}}$ is defined as

$$r_{i,j,t}^{\text{fol}} = \begin{cases} -P_2 & D_{i,j}^t \leq R_s \\ w_2(D_{i,j}^t - R_a) & R_s < D_{i,j}^t \leq R_a \\ 0 & D_{i,j}^t > R_a, \end{cases} \quad (11)$$

where $P_2 \gg 0$ is a large positive constant, and $w_2 > 0$ is a tuning parameter. Similarly, in order to prevent the follower i from conflicting with the k -th neighbor-intruders, $r_{i,k,t}^{\text{int}}$ is defined as

$$r_{i,k,t}^{\text{int}} = \begin{cases} -P_1 & D_{i,k}^t \leq R_s \\ w_1(D_{i,k}^t - R_a) & R_s < D_{i,k}^t \leq R_a \\ 0 & D_{i,k}^t > R_a. \end{cases} \quad (12)$$

III. METHODOLOGY

In this section, we propose a novel multi-agent actor-critic RL algorithm to solve the flocking with collision avoidance problem, is illustrated in Figure 1. We start by representing the dynamic dimensional observation of the follower via the designed population-invariant network architecture (Section III-A). We then detail the proposed spatial-temporal attention multi-agent actor-critic (STAAC) algorithm for policy optimization (Section III-B).

A. Population-Invariant Network Architecture

Since the UAV fleet is scalable and the number of intruders is varying, the dimension of each follower's observation is dynamic. To represent the dynamic dimensional observation to a fixed-size embedding, we design a novel spatial-temporal attention based population-invariant network architecture, as shown in Figure 2. We first introduce the spatial-temporal attention network. Afterwards, we describe the policy network (i.e., actor) and then the value network (i.e., critic).

1) *Spatial-Temporal Attention Network*: The first step in the spatial-temporal attention network is to cluster the entities in the environment into different groups. Then, we use the *local spatial attention* (LSA) module to separately compute the spatial embedding of each group (see Figure 2(c)). Afterwards, we design a *global temporal attention* (GTA) module to extract the temporal embedding of different groups, as indicated in Figure 2(e).

a) *Entity clustering*: The flocking with collision avoidance task involves different types of entities. Thus, it is more effective for agents to handle these entities separately due to their heterogeneities and differences [32]. To this end, we first cluster the entities within the observable range of the agent into different groups according to their characteristics and properties. From the perspective of each follower i , the observed entities can be partitioned into four groups: itself, the leader, the neighbor-followers, and the neighbor-intruders. Accordingly, we utilize the LSA module to treat them separately.

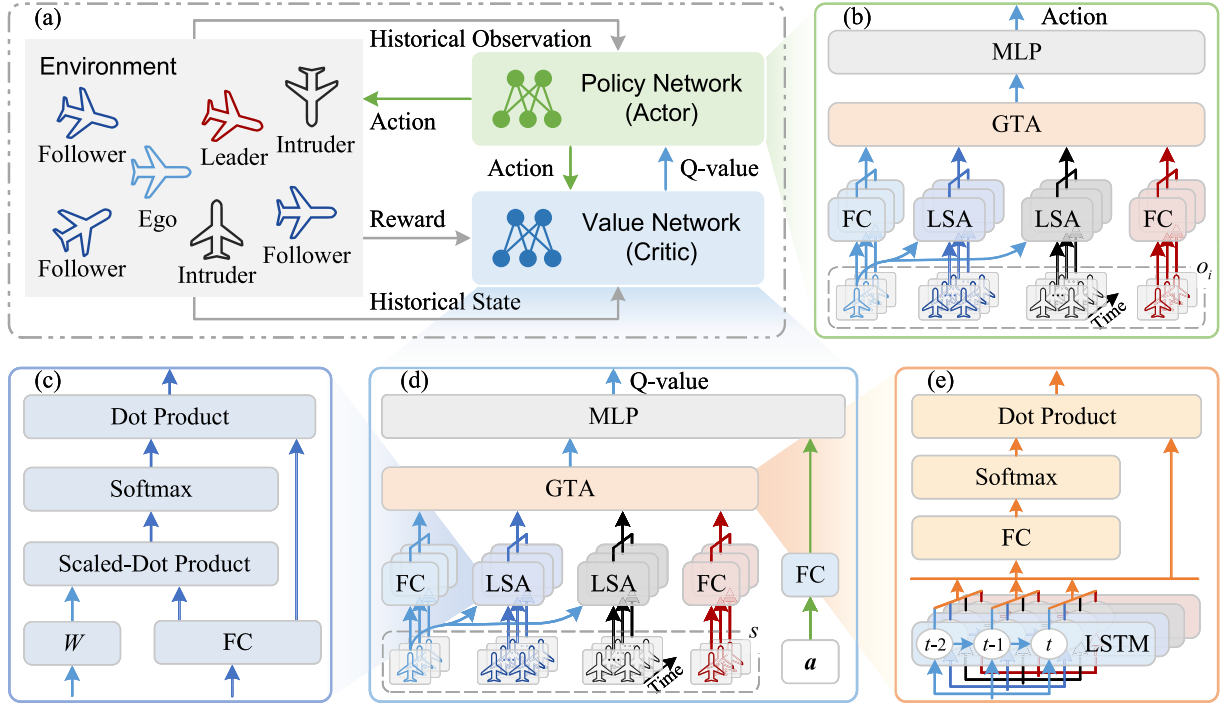


Fig. 2. Spatial-temporal attention based population-invariant network architecture. (a) Illustration of the interaction between the environment and the agents. (b) Policy network (actor). (c) Local spatial attention (LSA) module. (d) Value network (critic). (e) Global temporal attention (GTA) module.

b) Local spatial attention (LSA): The entities within the same group possess different spatial features. We extract the spatial features by distinguishing the importance of different entities [23]. Now we take the group of neighbor-followers as an example to reveal the operations of LSA. Mathematically, the spatial embedding of neighbor-followers at the time step t (i.e., $e_{i,t}^{\text{fol}}$) is defined as the aggregation of all neighbor-followers' states:

$$e_{i,t}^{\text{fol}} = \sum_{j \in \mathcal{F}_i^t} \alpha_{i,j,t}^{\text{fol}} \text{FC}(\xi_{j,t}^{\text{fol}}). \quad (13)$$

The spatial attention weight $\alpha_{i,j,t}^{\text{fol}}$ is determined by

$$\alpha_{i,j,t}^{\text{fol}} = \frac{\exp(\beta_{i,j,t}^{\text{fol}})}{\sum_j \exp(\beta_{i,j,t}^{\text{fol}})}, \quad (14a)$$

$$\beta_{i,j,t}^{\text{fol}} = \frac{(\xi_{i,t}^{\text{fol}})^T W_{\text{fol}}^T \text{FC}(\xi_{j,t}^{\text{fol}})}{\sqrt{d_{\text{fol}}}}, \quad (14b)$$

where $(\cdot)^T$ is the transpose operation, $\text{FC}(\cdot)$ is a fully-connected (FC) layer, W_{fol} is the parameter to learn, and d_{fol} is the dimension of $\text{FC}(\xi_{j,t}^{\text{fol}})$.

In the same way, we can calculate the aggregated spatial embedding for the neighbor-intruders at the time step t (i.e., $e_{i,t}^{\text{int}}$) as follows:

$$e_{i,t}^{\text{int}} = \sum_{k \in \mathcal{I}_i^t} \alpha_{i,k,t}^{\text{int}} \text{FC}(\xi_{k,t}^{\text{int}}), \quad (15a)$$

$$\alpha_{i,k,t}^{\text{int}} = \frac{\exp(\beta_{i,k,t}^{\text{int}})}{\sum_k \exp(\beta_{i,k,t}^{\text{int}})}, \quad (15b)$$

$$\beta_{i,k,t}^{\text{int}} = \frac{(\xi_{i,t}^{\text{fol}})^T W_{\text{int}}^T \text{FC}(\xi_{k,t}^{\text{int}})}{\sqrt{d_{\text{int}}}}. \quad (15c)$$

where d_{int} denotes the dimension of $\text{FC}(\xi_{k,t}^{\text{int}})$. We note that the dimension of each group's spatial embedding is invariant to the number of entities in the group.

When only one entity exists in a group, the spatial attention weight becomes a scalar of 1. This indicates that the aforementioned local spatial attention operations can be degenerated to a $\text{FC}(\cdot)$ function for the group with a single entity. Thus, the spatial embeddings of the leader and the follower i at the time step t (i.e., e_t^{lea} and e_t^{ego}) can be respectively expressed as

$$e_t^{\text{lea}} = \text{FC}(\xi_t^{\text{lea}}), \quad (16)$$

$$e_{i,t}^{\text{ego}} = \text{FC}(\xi_{i,t}^{\text{fol}}). \quad (17)$$

c) Global temporal attention (GTA): After the LSA modules, we employ a GTA module to selectively aggregate the temporal features of different groups. To ameliorate the partial observability issue, keeping track of historical observations for decision-making is essential for decision-making [33]. In the flocking with collision avoidance task, we stack the past four frame observations for each follower. Due to the different roles and contributions of the different frames, explicitly distinguishing the importance of the frames is beneficial for the followers to achieve their goals [31]. For this purpose, we integrate recurrent neural networks with the temporal attention mechanism into the GTA module.

We now introduce the specific operations in the GTA module. We first use long short-term memory (LSTM) networks to separately extract temporal features from historical observations for each group. Specifically, for the group of

the leader, its temporal features extracted by an LSTM layer $\text{LSTM}(\cdot)$ can be expressed as follows:

$$\begin{aligned} h_{\tau}^{\text{lea}} &= \text{LSTM}(e_{\tau}^{\text{lea}}, h_{\tau-1}^{\text{lea}}), \\ &= g_{\tau} \tanh(C_{\tau}), \quad \tau \in [t-3, t], \end{aligned} \quad (18)$$

where

$$g_{\tau} = \text{sigmoid}\left(W_{ge} e_{\tau}^{\text{lea}} + W_{gh} h_{\tau-1}^{\text{lea}} + b_g\right), \quad (19)$$

$$C_{\tau} = f_{\tau} C_{\tau-1} + l_{\tau} \tilde{C}_{\tau}, \quad (20)$$

$$f_{\tau} = \text{sigmoid}\left(W_{fe} e_{\tau}^{\text{lea}} + W_{fh} h_{\tau-1}^{\text{lea}} + b_f\right), \quad (21)$$

$$l_{\tau} = \text{sigmoid}\left(W_{le} e_{\tau}^{\text{lea}} + W_{lh} h_{\tau-1}^{\text{lea}} + b_l\right), \quad (22)$$

$$\tilde{C}_{\tau} = \tanh\left(W_{Ce} e_{\tau}^{\text{lea}} + W_{Ch} h_{\tau-1}^{\text{lea}} + b_C\right), \quad (23)$$

$$h_{t-4}^{\text{lea}} = \mathbf{0}. \quad (24)$$

Here $W_{ge}, W_{gh}, W_{fe}, W_{fh}, W_{le}, W_{lh}, W_{Ce}, W_{Ch}$ denote the weights of the LSTM layer, and b_g, b_f, b_l, b_C are the corresponding biases. Similarly, the temporal features of the other three groups can be represented as

$$h_{i,\tau}^{\text{ego}} = \text{LSTM}(e_{i,\tau}^{\text{ego}}, h_{i,\tau-1}^{\text{ego}}), \quad \tau \in [t-3, t], \quad (25)$$

$$h_{i,\tau}^{\text{fol}} = \text{LSTM}(e_{i,\tau}^{\text{fol}}, h_{i,\tau-1}^{\text{fol}}), \quad \tau \in [t-3, t], \quad (26)$$

$$h_{i,\tau}^{\text{int}} = \text{LSTM}(e_{i,\tau}^{\text{int}}, h_{i,\tau-1}^{\text{int}}), \quad \tau \in [t-3, t], \quad (27)$$

where $h_{i,t-4}^{\text{ego}} = \mathbf{0}, h_{i,t-4}^{\text{fol}} = \mathbf{0}, h_{i,t-4}^{\text{int}} = \mathbf{0}$.

Concatenating the temporal features of the four groups, the global temporal embedding of the agent i at the time step τ can be defined as

$$h_{i,\tau}^{\text{glo}} = \left(h_{\tau}^{\text{lea}} \parallel h_{i,\tau}^{\text{ego}} \parallel h_{i,\tau}^{\text{fol}} \parallel h_{i,\tau}^{\text{int}}\right), \quad \tau \in [t-3, t], \quad (28)$$

where $\parallel$ represents the concatenation operation.

We then use the temporal attention mechanism [30], [31] to identify which time slot the agent should focus on more. Mathematically, the temporal attention weight $\alpha_{i,\tau}^{\text{glo}}$ can be calculated as follows:

$$\alpha_{i,\tau}^{\text{glo}} = \frac{\exp(\beta_{i,\tau}^{\text{glo}})}{\sum_{\tau} \exp(\beta_{i,\tau}^{\text{glo}})}, \quad (29a)$$

$$\beta_{i,\tau}^{\text{glo}} = \text{FC}(h_{i,\tau}^{\text{glo}}), \quad (29b)$$

The global temporal attention embedding of the agent i at the time step t can be defined as:

$$v_i^t = \sum_{\tau=t}^{t-3} \alpha_{i,\tau}^{\text{glo}} h_{i,\tau}^{\text{glo}}. \quad (30)$$

2) *Policy Network*: We now define the structure of the policy network. As shown in Figure 2(b), we first use the spatial-temporal attention network to represent the agent's observation:

$$v_i^t = \text{STAN}(o_i^t), \quad (31)$$

where $\text{STAN}(\cdot)$ denotes the spatial-temporal attention network defined in Section III-A1. We then map the global temporal attention embedding v_i^t to the action space:

$$a_i^t = \text{MLP}(v_i^t), \quad (32)$$

where $\text{MLP}(\cdot)$ is a two-layer multi-layer perceptron (MLP).

3) *Value Network*: The overall structure of the value network is depicted in Figure 2(d). The structure of the value network is similar to the policy network, except that the value network adds an FC layer as an action encoder to process the joint action $\mathbf{a}^t := (a_1^t, \dots, a_n^t)$. Formally, we first use the spatial-temporal attention network to represent the environment state:

$$z_i^t = \text{STAN}(s^t). \quad (33)$$

The action encoder is defined as

$$c_i^t = \text{FC}(\mathbf{a}^t). \quad (34)$$

Afterwards, we concatenate the global temporal attention embedding z_i^t with the action embedding c_i^t , and then pass it to an MLP to approximate the Q-value of the agent i :

$$q_i^t = \text{MLP}(z_i^t \parallel c_i^t). \quad (35)$$

B. Learning Algorithm

We now present our STAAC algorithm in detail to learn the scalable flocking with collision avoidance policy. Consistent with well-established multi-agent RL algorithms, e.g., MADDPG [24], STAAC follows the paradigm of centralized training and decentralized execution. Each agent learns its own centralized critic by concatenating the states and actions of all agents as inputs. In contrast, the actors for all agents are decentralized, relying only on their own local observations.

To improve learning efficiency and scalability, STAAC introduces the parameter sharing mechanism [25]. One reason is that the agents in our flocking task are homogeneous, so that different agents are able to share the same control policy. Consequently, STAAC only needs to learn a shared policy and a centralized critic for all agents. Besides, we use a shared experience replay buffer to store the experiences collected by all agents simultaneously. Specifically, the experience is stored in the form of a tuple $(s^t, \mathbf{o}^t, \mathbf{a}^t, \mathbf{r}^t, \mathbf{o}^{t+1}, s^{t+1})$. Here $\mathbf{o}^t := (o_1^t, \dots, o_n^t)$ denotes the concatenation of all agents' observation at the time step t . Similarly, $\mathbf{a}^t := (a_1^t, \dots, a_n^t)$, $\mathbf{r}^t := (r_1^t, \dots, r_n^t)$ represent the actions and rewards of all agents at the time step t , respectively.

We update the policy and the critic by randomly sampling a minibatch of experiences from the replay buffer. Mathematically, let $\mathcal{D}$ denote the replay buffer, and μ represent the policy network. The policy's parameter θ^{μ} is updated with the gradient [24]:

$$\begin{aligned} \nabla_{\theta^{\mu}} \mathcal{J}(\theta^{\mu}) &= \mathbb{E}_{(s^t, \mathbf{o}^t, \mathbf{a}^t) \sim \mathcal{D}} \left[\frac{1}{n} \sum_{i=1}^n \left(\nabla_{\theta^{\mu}} \mu(o_i^t) \right. \right. \\ &\quad \left. \left. \times \nabla_{a_i^t} Q_1(s^t, a_1^t, \dots, a_n^t) \Big|_{a_i^t = \mu(o_i^t)} \right) \right], \end{aligned} \quad (36)$$

where Q_1 denotes the critic network. The critic's parameter θ_1^Q is optimized by minimizing the loss [24]:

$$\mathcal{L}(\theta_1^Q) = \mathbb{E}_{(s^t, \mathbf{a}^t, \mathbf{r}^t, \mathbf{o}^{t+1}, s^{t+1}) \sim \mathcal{D}} \left[\frac{1}{n} \sum_{i=1}^n \left(Q_1(s^t, \mathbf{a}^t) - y_i^t \right)^2 \right], \quad (37)$$

where y_i^t denotes the target Q-value.

To reduce the overestimation bias, we incorporate the clipped double Q-learning [26] trick into our STAAC algorithm. Specifically, we additionally learn an extra critic Q_2 with parameter θ_2^Q in the same way as Q_1 . The minimum of the two Q-values estimated by Q_1 and Q_2 is used to form the target Q-value:

$$y_i^t = r_i^t + \gamma \min_{l=1,2} Q_l^-(s^{t+1}, \tilde{a}_1^{t+1}, \dots, \tilde{a}_n^{t+1}). \quad (38)$$

The smoothed target action $\tilde{a}_i^{t+1}$ is defined as [34]

$$\begin{aligned} \tilde{a}_j^{t+1} &= \mu^-(o_j^{t+1}) + \epsilon_a, \\ \epsilon_a &\sim \text{clip}(\mathcal{N}(0, \sigma_a^2), -c, c), \end{aligned} \quad (39)$$

where σ_a^2 and c denote the variance and boundary of Gaussian noise ϵ_a . We note that μ^- and $Q_{l=1,2}^-$ denote target networks of the policy and the two critics, receptively. Following MADDPG [24], the parameters of the target networks θ^{μ^-} and $\theta_{l=1,2}^{Q^-}$ are updated as follows:

$$\begin{aligned} \theta^{\mu^-} &\leftarrow \lambda \theta^{\mu} + (1 - \lambda) \theta^{\mu^-}, \\ \theta_l^{Q^-} &\leftarrow \lambda \theta_l^Q + (1 - \lambda) \theta_l^{Q^-}, \end{aligned} \quad (40)$$

where $\lambda \ll 1$ is the soft update rate.

C. Summary and Analysis

The overall workflow of the proposed STAAC algorithm is summarized in Algorithm 1. The development of the STAAC algorithm lays the foundation for learning the flocking control policy for a scalable fleet of UAV agents. Given that the UAV agents are homogeneous, STAAC adopts the parameter sharing mechanism to learn a shared control policy. After training, the UAV agents determine their own actions based on the shared policy, relying solely on their individual local observations. This allows the UAV agents to maintain decentralized control during operation, which is crucial for scalability in terms of fleet size. Another contribution to improving scalability is the population-invariant neural network architecture. This architecture incorporates the local spatial attention and global temporal attention mechanisms, facilitating representation learning for dynamic dimensional observations. These attention mechanisms enhance the learning and coordination abilities of UAV agents, enabling them to focus on relevant information and enhance performance in dynamic environments. Moreover, they enable flocking with collision avoidance policies to adapt to changes in fleet size and intruder numbers, demonstrating scalability and adaptability to different scenarios by adeptly efficiently varying amounts of information.

The time and space complexity of the proposed STAAC algorithm are analyzed as follows. It is well-known that the computational complexity of deep learning-based algorithms can be considered from two distinct phases: model training and model inference. During the training phase, network parameters are iteratively updated based on a loss function, often requiring large amounts of data. For the STAAC algorithm, the most time-consuming operations are the updates of the actor and critic networks. Therefore, the training time complexity

Algorithm 1 STAAC

Input: Replay buffer maximum capacity N , maximum time step per episode N_s , batch size N_b , update frequency C ;
Output: Well-trained policy network parameters $\theta^{\mu*}$;

- 1: Initialize network parameters θ^{μ} and $\theta_{l=1,2}^Q$ for policy and two critics randomly;
- 2: Initialize target networks: $\theta^{\mu^-} \leftarrow \theta^{\mu}$; $\theta_{l=1,2}^{Q^-} \leftarrow \theta_{l=1,2}^Q$;
- 3: Initialize replay buffer $\mathcal{D}$ with maximum capacity N ;
- 4: **for** episode = 1 to maximum training episode **do**
- 5: Initialize the states of all entities randomly;
- 6: Receive initial state s^1 and initial observations $\mathbf{o}^1$;
- 7: **for** $t = 1$ to N_s **do**
- 8: Select action with current policy and exploration noise: $a_i^t = \mu(o_i^t) + \epsilon_e$, $\epsilon_e \sim \mathcal{N}(0, \sigma_e^2)$;
- 9: Execute joint actions and update system state s^{t+1} ;
- 10: Compute immediate reward r_i^t with (9);
- 11: Receive new observations $\mathbf{o}^{t+1}$;
- 12: Store $(s^t, \mathbf{o}^t, \mathbf{a}^t, r^t, \mathbf{o}^{t+1}, s^{t+1})$ into $\mathcal{D}$;
- 13: $s^t \leftarrow s^{t+1}$; $\mathbf{o}^t \leftarrow \mathbf{o}^{t+1}$;
- 14: Sample a minibatch of N_b tuples from $\mathcal{D}$;
- 15: Update $\theta_{l=1,2}^Q$ by minimizing the loss of (37);
- 16: **if** $t \bmod C = 0$ **then**
- 17: Update θ^{μ} according to gradient of (36);
- 18: Update target network parameters θ^{μ^-} and $\theta_{l=1,2}^{Q^-}$ with (40);
- 19: **end if**
- 20: **end for**
- 21: **end for**

of the STAAC algorithm can be expressed as $O(N_{\text{episodes}} \cdot N_s \cdot N_b \cdot (2m_Q + m_{\mu}/C))$. Here N_{episodes} is the number of training episodes; N_s is the number of time steps per episode; N_b is the minibatch size; m_Q is the number of parameters in the critic networks; m_{μ} is the number of parameters in the actor network; C is the update frequency for the actor network. This expression indicates that the training complexity grows linearly with the number of episodes, time steps per episode, and the size of the networks. Typically, the time training complexity is high as numerous iterations are required for convergence. In contrast, during the inference phase, the model directly outputs actions based on input observations, without the need for backpropagation or parameter updates. Thus, the inference time complexity is determined by the size of the policy network, which is denoted as $O(m_{\mu})$. It is evident that the inference phase is computationally lightweight compared to the training phase. Once the model training is completed, inferences can be made within millisecond timescales. Regarding the space complexity of the STAAC algorithm, we note that the total space complexity is primarily determined by the replay buffer and the storage of network parameters. The replay buffer can store up to N transitions and if the size of each transition is denoted as T_{size} , the space complexity for the replay buffer is $O(N \cdot T_{\text{size}})$. Additionally, the space required for storing the parameters of the actor network and two critic networks is $O(m_{\mu} + 2m_Q)$. Therefore, the total space complexity of STAAC is denoted as $O(N \cdot T_{\text{size}} + m_{\mu} + 2m_Q)$.

TABLE I
PARAMETER SETTINGS

Name	Value	Name	Value
R_c	100 m	R_a	50 m
R_s	15 m	σ_v^2	0.5
σ_x^2	0.5	σ_y^2	0.5
$\bar{\omega}_n$	6.3	σ_ω^2	0.5
$\bar{\zeta}$	0.55	σ_ζ^2	0.05
$v_{\max}$	18 m/s	$v_{\min}$	12 m/s
$\omega_{\max}$	$\pi/12$	$u_{\max}$	1 m/s
w_1	5	w_2	10
$P1$	1000	$P2$	2000
N_b	32	N_s	60
N	50000	γ	0.95
λ	0.01	C	2
σ_a^2	0.2	c	0.5

IV. SIMULATION RESULTS

In this section, we validate the proposed approach in numerical and hardware-in-the-loop (HITL) simulations.

A. Implementation Details

In the LSA modules, the FC layers for the leader, ego-follower, neighbor-followers, and neighbor-intruders had 64, 64, 128, and 128 neurons, respectively. All the FC layers were activated by the rectified linear unit (ReLU) function. The learnable parameter $W_{\text{fol}} \in \mathbb{R}^{128 \times 4}$ and $W_{\text{int}} \in \mathbb{R}^{128 \times 4}$. In the GTA module, the LSTM layers for the leader, ego-follower, neighbor-followers, and neighbor-intruders had 64, 64, 128, and 128 hidden units, respectively. For the actor and the critic, the MLP networks both contained two FC layers. The hidden layers of them had 256 neurons, but the output layers had 2 and 1 neuron(s), which were activated by the hyperbolic tangent (tanh) and linear functions, respectively. The learning rates for the policy and the critic were set to 0.0001 and 0.001, respectively. The maximum training episode was set to 5000, during which the exploration rate ϵ_e was annealed exponentially from 0.5 to 0.05. The empirical values of other parameters are detailed in Table I.

B. Numerical Simulation Results

1) *Training Results:* We first compared our STAAC algorithm with i) generic multi-agent RL algorithms (including MADDPG [24], MATD3 [26], and HAMA [32]), ii) our previously proposed collision-free flocking method API-MADDPG [23], and iii) existing state-of-the-art multi-agent RL based collision avoidance methods (i.e., BCDDPG [20] and LSTM-DDQN [35]). To enable the learned policies to adapt to the changes in the scale of UAV fleets, the three generic multi-agent RL algorithms also employed the parameter sharing [25] mechanism as in STAAC and API-MADDPG. Since MADDPG, MATD3, and LSTM-DDQN cannot handle the dynamic dimensional observation, only the two closest neighbor-followers and the closest neighbor-intruders were considered for decision-making. We used a two-layer MLP with 256 and 256 neurons as the actor and critic networks,

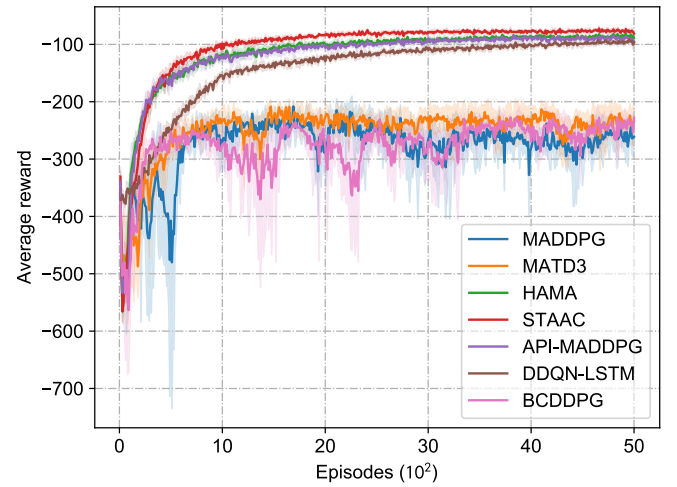


Fig. 3. Learning curves of MDDPG, MATD3, HAMA, STAAC, API-MADDPG, BCDDPG, and LSTM-DDQN. Note that the reward values are averaged every 10 episodes.

respectively. In the LSTM-DDQN method, we discretized the action space into 9 candidate actions and employed two branches equipped with LSTM networks to independently process the states of followers and intruders.

We adopted the seven algorithms to learn the flocking with collision avoidance policies separately. As for the environment settings, ten followers flew with the leader in a rectangular area of 1200 m $\times$ 800 m. Fifteen intruders were randomly distributed in the task environment. The locations and intentions of the intruders, as well as the predefined path of the leader, were randomly initialized in each training episode.

The average reward received by the followers with the seven algorithms is shown in Figure 3. Intuitively, the seven learning curves grow rapidly at the initial stage. After about 1000 training episodes (about 4 hours of wall-clock time with NVIDIA RTX 3080 laptop GPU), the seven curves gradually converge. We note that the convergence values are very different. The final rewards of the three attention based learning algorithms, i.e., HAMA, API-MADDPG, and STAAC, are significantly higher than those of MDDPG and MATD3. This result suggests that integrating attention mechanisms into multi-agent RL is very useful for improving performance. HAMA and STAAC achieve a higher average reward than API-MADDPG. This means that aggregating different groups of entity features separately is more effective than dealing with them together. In addition, STAAC obtains a higher average reward than HAMA, showing the power of the proposed global temporal attention mechanism. In comparison to existing multi-agent RL based collision avoidance methods, namely BCDDPG and LSTM-DDQN, our STAAC algorithm achieves the highest final reward with the fastest convergence speed. The above results demonstrate the superior performance of the proposed STAAC algorithm, surpassing not only the four generic multi-agent RL algorithms but also the two state-of-the-art multi-agent RL based collision avoidance methods.

2) *Testing Results:* After training, we tested the flocking with the collision avoidance policy learned with our STAAC algorithm in a typical task. Ten intruders were rel-

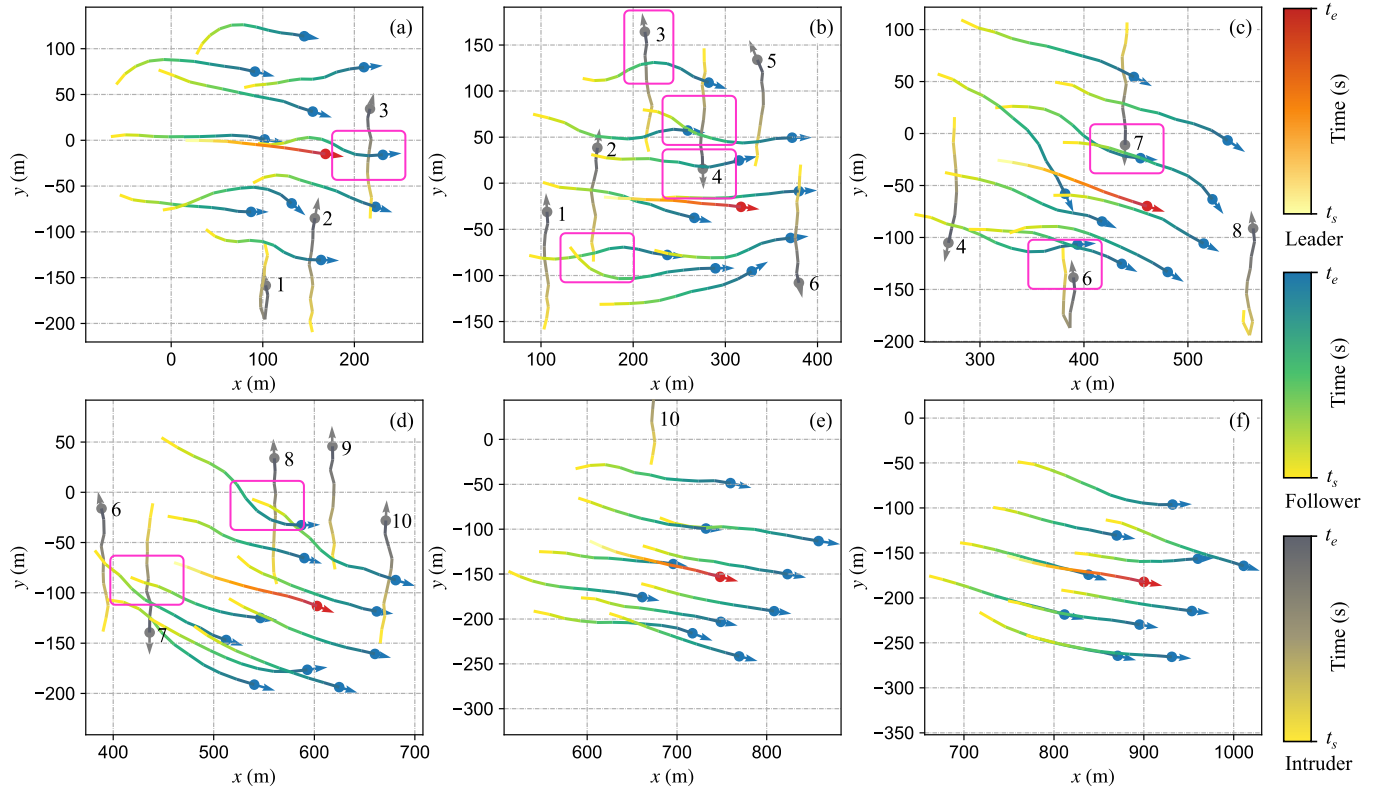


Fig. 4. Trajectories of the ten followers using the learned STAAC policy to flock with the leader and avoid collisions with the ten intruders. The final positions and movement directions of the entities are marked on the trajectories with dots and arrows. (a) $t = 0 \sim 10$ s. (b) $t = 10 \sim 20$ s. (c) $t = 20 \sim 30$ s. (d) $t = 30 \sim 40$ s. (e) $t = 40 \sim 50$ s. (f) $t = 50 \sim 60$ s.

actively uniformly distributed in the rectangular task area of $1200\text{m} \times 400\text{m}$. The leader and the intruders flew along their pre-planned paths. The leader's heading rate and forward acceleration were randomly sampled from $-5 \sim 5^\circ$ and $-1 \sim 1\text{m/s}$ at each time step, respectively. The speeds of the intruders were randomly sampled from $10 \sim 15\text{m/s}$ at each time step. Ten followers employed the learned STAAC policy to follow the leader lasting for 60 time steps.

The trajectories of the UAV fleet and the intruders in the testing are depicted in Figure 4. As shown, the followers can keep up with the leader and successfully avoid colliding with each other, whether the leader flies in the cluttered space (see Figure 4(a) ~ (d)) or in the free space (see Figure 4(e) ~ (f)). At the same time, the followers are able to keep away from the intruders, avoiding potential conflicts, as shown by the purple rectangles in Figure 4.

The flocking performance of the learned STAAC policy in the testing is shown in Figure 5. As can be seen, the distances from the leader to the followers (ρ) are maintained in the range of 50 to 100 meters most of the time. In addition, the minimum distance among the followers ($D_{\min}^{\text{fol}}$) as well as the minimum distance between the intruders and the followers ($D_{\min}^{\text{int}}$) are both large than the minimum safety radius of followers $R_s = 15\text{m}$ at each time step. This means that there were no collisions happened among the followers or between any follower and the intruders throughout the testing.

3) *Generalization Results:* To validate the scalability of the proposed STAAC algorithm, we directly generalized the learned flocking with collision avoidance policies to dynamic

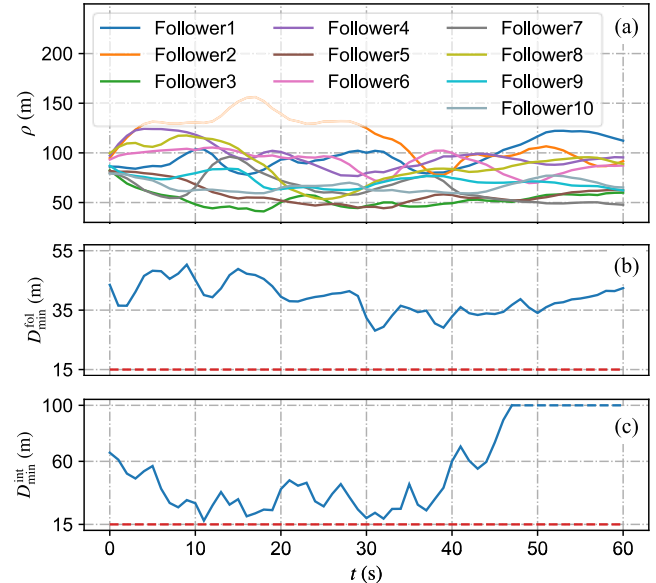


Fig. 5. Performance of the ten followers using the learned STAAC policy to flock with the leader and avoid collisions with the ten intruders. (a) The distances from the leader to the followers. (b) The minimum distance among the followers. (c) The minimum distance between the followers and the intruders (Note that the upper limit is fixed at 100 m for better visibility).

environments with different numbers of followers and intruders. We also compared our STAAC algorithm with two classical non-learning collision avoidance methods, i.e., the artificial potential field (APF) method [36] and the optimal reciprocal collision avoidance (ORCA) method [37].

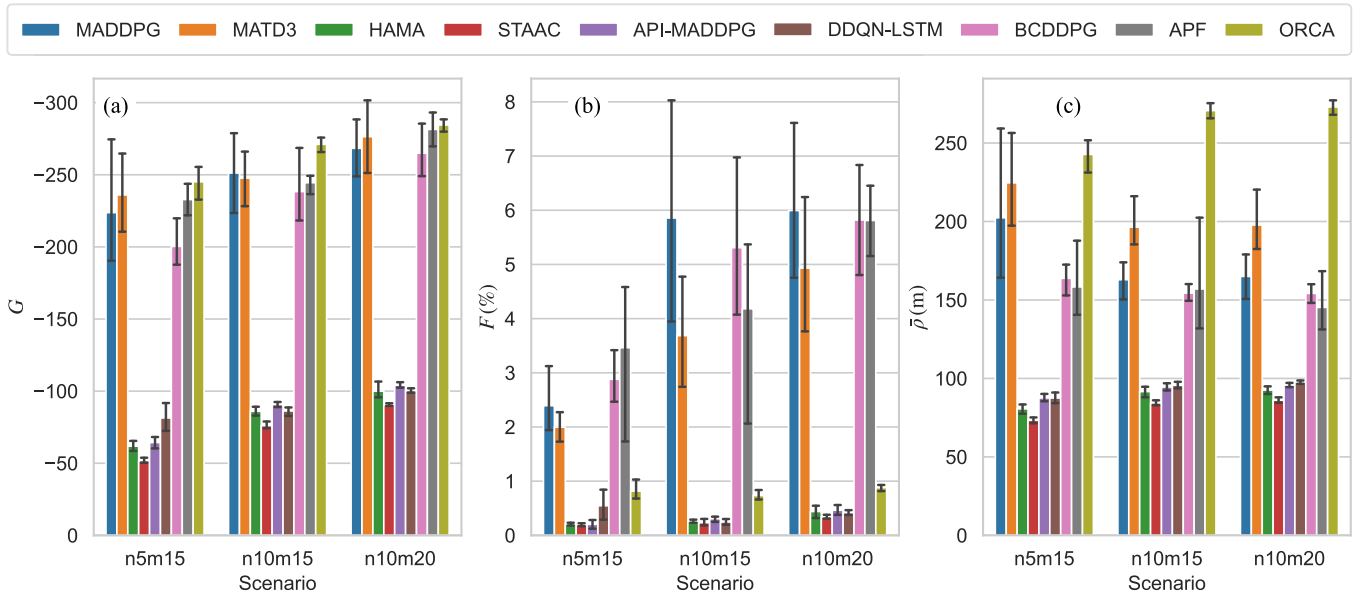


Fig. 6. Bar charts of generalization performance of the five learned flocking with collision avoidance policies. (a) Average reward (Note that the vertical axis is inverted). (b) Collision rate. (c) Average distance between the followers and the leader.

TABLE II

GENERALIZATION RESULTS OF THE LEARNED FLOCKING WITH COLLISION AVOIDANCE POLICIES IN DYNAMIC SCENARIOS WITH DIFFERENT NUMBERS OF FOLLOWERS AND INTRUDERS

Method	n5m15			n10m15			n10m20		
	G	F (%)	$\bar{\rho}$ (m)	G	F (%)	$\bar{\rho}$ (m)	G	F (%)	$\bar{\rho}$ (m)
MADDPG [24]	-223.76 ± 49.18	2.39 ± 0.70	202.39 ± 58.39	-251.21 ± 32.66	5.86 ± 2.37	162.93 ± 14.07	-268.34 ± 22.89	6.00 ± 1.73	165.01 ± 18.01
MATD3 [26]	-236.05 ± 30.95	2.00 ± 0.31	224.65 ± 33.57	-247.67 ± 23.53	3.69 ± 1.15	196.37 ± 19.58	-276.43 ± 29.08	4.93 ± 1.45	197.75 ± 22.10
HAMA [32]	-61.83 ± 4.16	0.21 ± 0.03	80.62 ± 3.55	-85.99 ± 3.60	0.26 ± 0.03	91.62 ± 4.01	-99.99 ± 6.44	0.44 ± 0.14	92.61 ± 2.81
STAAC (Ours)	-51.81 ± 2.09	0.20 ± 0.03	73.37 ± 2.05	-76.09 ± 2.89	0.24 ± 0.07	84.41 ± 1.95	-90.73 ± 1.06	0.34 ± 0.04	86.12 ± 2.03
API-MADDPG [23]	-64.36 ± 4.57	0.20 ± 0.10	87.57 ± 2.69	-90.73 ± 2.00	0.31 ± 0.06	94.55 ± 2.56	-104.11 ± 2.24	0.46 ± 0.11	95.77 ± 1.61
LSTM-DDQN [35]	-81.40 ± 10.90	0.55 ± 0.31	87.53 ± 3.68	-86.25 ± 3.62	0.25 ± 0.06	95.67 ± 2.49	-100.41 ± 1.79	0.42 ± 0.05	97.73 ± 1.27
BCDDPG [20]	-200.36 ± 20.13	2.88 ± 0.54	163.80 ± 10.94	-238.36 ± 31.85	5.31 ± 1.74	154.39 ± 6.17	-264.97 ± 21.81	5.82 ± 1.15	154.20 ± 7.15
APF [36]	-234.82 ± 10.61	4.20 ± 0.40	144.84 ± 4.53	-246.62 ± 3.79	5.19 ± 0.22	133.07 ± 3.71	-279.79 ± 12.38	6.04 ± 0.45	136.04 ± 5.87
ORCA [37]	-245.11 ± 13.22	0.82 ± 0.22	242.76 ± 11.52	-271.12 ± 6.04	0.74 ± 0.10	270.65 ± 5.57	-284.49 ± 5.15	0.88 ± 0.07	273.01 ± 5.43

Specifically, we evaluated the seven learned policies and the two non-learning methods in three different scenarios, i.e., n5m15, n10m15, and n10m20, where n denotes the number of followers and m represents the number of intruders. These evaluations consisted of 100 episodes, each of which lasted for 60 time steps. We evaluated the performance of flocking with collision avoidance methods in three aspects: the average reward of each follower at each time step G , the average collision rate F , and the average distance between the leader and the followers $\bar{\rho}$.

Figure 6 and Table II report the zero-shot generalization results on the dynamic scenarios with different numbers of followers and intruders. As shown, the flocking policies learned using the seven algorithms are able to adapt to changes in the size of UAV fleets. This is primarily attributed to the parameter sharing mechanism. The performance of the APF and ORCA methods is comparable to that of MADDPG, MATD3, and BCDDPG, while the other four algorithms (HAMA, STAAC, API-MADDPG, and LSTM-DDQN) exhibit

significantly superior performance compared to APF and ORCA. These results indicate the capability of multi-agent RL based methods can reach or even surpass classical collision avoidance methods. With the power of distinguishing the importance levels of different entities through the attention mechanism or relative distance-based ordering, it is not surprising that the three attention based algorithms (i.e., HAMA, API-MADDPG, and STAAC) along with the LSTM-DDQN algorithm significantly outperform MADDPG, MATD3, and BCDDPG. HAMA and STAAC perform better in terms of the three metrics compared with API-MADDPG. This result confirms the contribution of entity clustering in the representation learning of the actor and the critic. STAAC achieves better performance than HAMA, particularly in terms of collision rates. As the number of entities increases, the advantage of STAAC becomes more apparent. In the n5m15 scenario, the collision rate of the STAAC is only 4.76% lower than that of the HAMA. In the n10m15 and n10m20 scenarios, this number becomes 7.69% and 22.73%, respectively. This result

means that the proposed spatial-temporal attention network is helpful in solving the reinforcement learning problem involving different types of entities. Overall, we can conclude that the proposed STAAC algorithm outperforms the seven baselines and achieves the best performance in terms of the three metrics, regardless of the size of UAV fleets and the number of intruders.

4) *Ablation Studies*: We then conducted ablation experiments in various scenarios to investigate the effectiveness of different components in STAAC. Specifically, we separately removed the LSA modules and the GTA module from STAAC, and then used the remaining components to learn the flocking with collision avoidance policies. TAAC only employed the GTA modules, we replaced the LSA module with the mean embedding of the different entities in each group. In contrast, SAAC only applied the LSA modules, treating the features of the four time slots equally. We kept the parameter settings the same as in Section IV-A and the environment configurations the same as in Section IV-B1. After training, we evaluated the learned flocking policies using the three algorithms in six different scenarios, i.e., n5m10, n5m15, n5m20, n10m10, n10m15, and n10m20. These evaluations consisted of 100 episodes, each of which lasted for 60 time steps.

The results of the ablation experiments are presented in Figure 7 and Table III. Intuitively, TAAC achieves the worst performance, and SAAC and STAAC perform significantly better than TAAC. This result shows that identifying the importance of different entities is very efficient for performance improvement than treating all entities equally. STAAC and SAAC have similar performance, but STAAC is slightly better. In the six testing scenarios, the flocking performance of STAAC is superior to that of SAAC in terms of the three metrics, i.e., average reward G , collision rate F , and average distance $\bar{\rho}$. Especially in the scenario n10m20, STAAC exhibits a 29.17% reduction compared to SAAC, resulting in a collision rate of 0.34%. This result means that the global temporal attention mechanism also plays an important role in policy learning, distinguishing the importance of different time slots is beneficial for followers to achieve their goals. To sum up, the above results show consistent improvement and stability of the proposed STAAC algorithm in various scenarios, indicating reliable performance and good generalization ability. We can conclude that both of the components are important for our STAAC algorithm, and spatial attention is more useful than temporal attention. By jointly cascading the LSA and GTA modules together, the proposed STAAC algorithm achieves the best performance.

C. HITL Experiment Results

We further evaluated the learned STAAC flocking policy in a high-fidelity hardware-in-the-loop (HITL) simulation system [16], [28]. Each UAV node in the HITL simulation system consisted of an X-plane 10 flight simulator, an onboard computer running the Ubuntu 16.04 LTS operating system and the robot operating system (ROS), a Pixhawk autopilot with the PX4 flight stack, and the QGroundControl. All UAV nodes

TABLE III
RESULTS OF ABLATION EXPERIMENTS IN DIFFERENT SCENARIOS
USING THE THREE LEARNED FLOCKING WITH COLLISION
AVOIDANCE POLICIES

Scenario	Method	G	F (%)	$\bar{\rho}$ (m)
n5m10	TAAC	-88.97 $\pm$ 5.78	0.93 $\pm$ 0.17	93.26 $\pm$ 4.22
	SAAC	-41.58 $\pm$ 2.47	0.10 $\pm$ 0.02	71.25 $\pm$ 1.89
	STAAC	-40.04 $\pm$ 3.00	0.08 $\pm$ 0.04	70.56 $\pm$ 1.94
n5m15	TAAC	-114.69 $\pm$ 5.18	1.74 $\pm$ 0.14	92.57 $\pm$ 4.52
	SAAC	-52.84 $\pm$ 2.65	0.24 $\pm$ 0.03	73.87 $\pm$ 2.73
	STAAC	-51.81 $\pm$ 2.09	0.20 $\pm$ 0.03	73.37 $\pm$ 2.05
n5m20	TAAC	-139.23 $\pm$ 4.42	2.43 $\pm$ 0.20	92.85 $\pm$ 4.08
	SAAC	-67.98 $\pm$ 3.65	0.44 $\pm$ 0.09	74.60 $\pm$ 1.89
	STAAC	-65.46 $\pm$ 2.82	0.28 $\pm$ 0.03	74.50 $\pm$ 1.75
n10m10	TAAC	-114.36 $\pm$ 2.72	1.11 $\pm$ 0.17	104.13 $\pm$ 2.96
	SAAC	-63.77 $\pm$ 3.19	0.12 $\pm$ 0.04	83.48 $\pm$ 2.05
	STAAC	-62.48 $\pm$ 2.47	0.11 $\pm$ 0.03	82.43 $\pm$ 1.76
n10m15	TAAC	-138.72 $\pm$ 3.52	1.80 $\pm$ 0.18	103.39 $\pm$ 2.05
	SAAC	-81.01 $\pm$ 3.62	0.24 $\pm$ 0.07	85.57 $\pm$ 2.64
	STAAC	-76.09 $\pm$ 2.89	0.23 $\pm$ 0.07	84.41 $\pm$ 1.95
n10m20	TAAC	-166.92 $\pm$ 3.30	2.63 $\pm$ 0.15	104.16 $\pm$ 2.71
	SAAC	-93.80 $\pm$ 3.70	0.48 $\pm$ 0.10	86.89 $\pm$ 1.66
	STAAC	-90.73 $\pm$ 1.06	0.34 $\pm$ 0.04	86.12 $\pm$ 2.03

were connected to a local area network and shared a ground control station.

In the HITL simulation system, five followers (UAV11 ~ 15 in Figure 8) employed the learned STAAC policy to follow the leader (UAV10 in Figure 8). Three intruders (UAV21 ~ 23 in Figure 8) were located in the mission area and flew along their pre-planned paths. The flight simulation lasted for 100 time steps (seconds). At each time step, each follower independently selected its action using the learned STAAC policy, while the leader determined its action according to the pre-planned path. We note that the flocking with collision avoidance policy used by each follower in the HITL simulation was learned from the numerical simulation without any parameter tuning.

Figure 8 presents the trajectories of the UAV fleet and the three intruders in the HITL simulation. As shown, the five followers are able to successfully keep up with the leader, even when the leader drastically changes its flight direction (see Figure 8(d) ~ (e)). More importantly, the followers can autonomously avoid the non-cooperative intruders while flocking with the leader, as shown by the yellow rectangles in Figure 8. For example, at $t = 85$ s and $t = 88$ s, UAV11 (Follower1) and UAV12 (Follower2) encounter UAV22 (Intruder2) in succession. Adhering to the learned STAAC policy, UAV11 performs a precise rightward maneuver (see Figure 8(f)), while UAV12 promptly executes a calculated leftward turn (see Figure 8(g)), successfully evading the trajectory of UAV22 and ensuring flight safety. The above results demonstrate that the followers using the learned STAAC policy can autonomously keep up with the leader while skillfully avoiding the non-cooperative intruders. It should be noted that in the HITL experiment, the average time for each UAV equipped with an

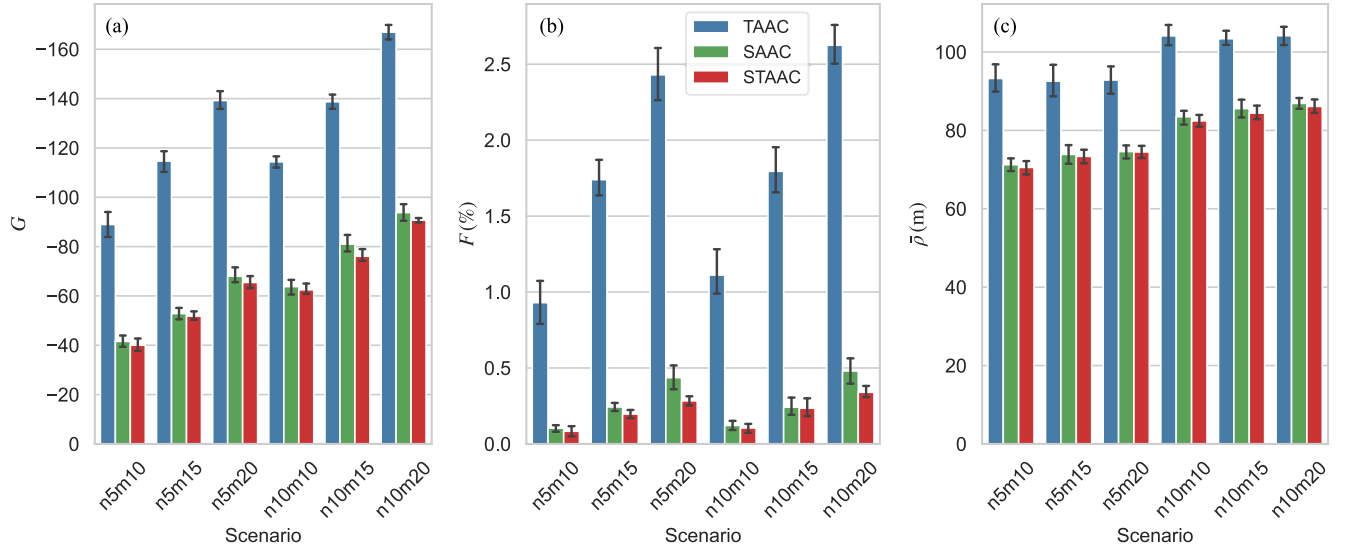


Fig. 7. Bar charts of ablation results of the learned flocking with collision avoidance policies in different scenarios. (a) Average reward (Note that the vertical axis is inverted). (b) Collision rate. (c) Average distance between the followers and the leader.

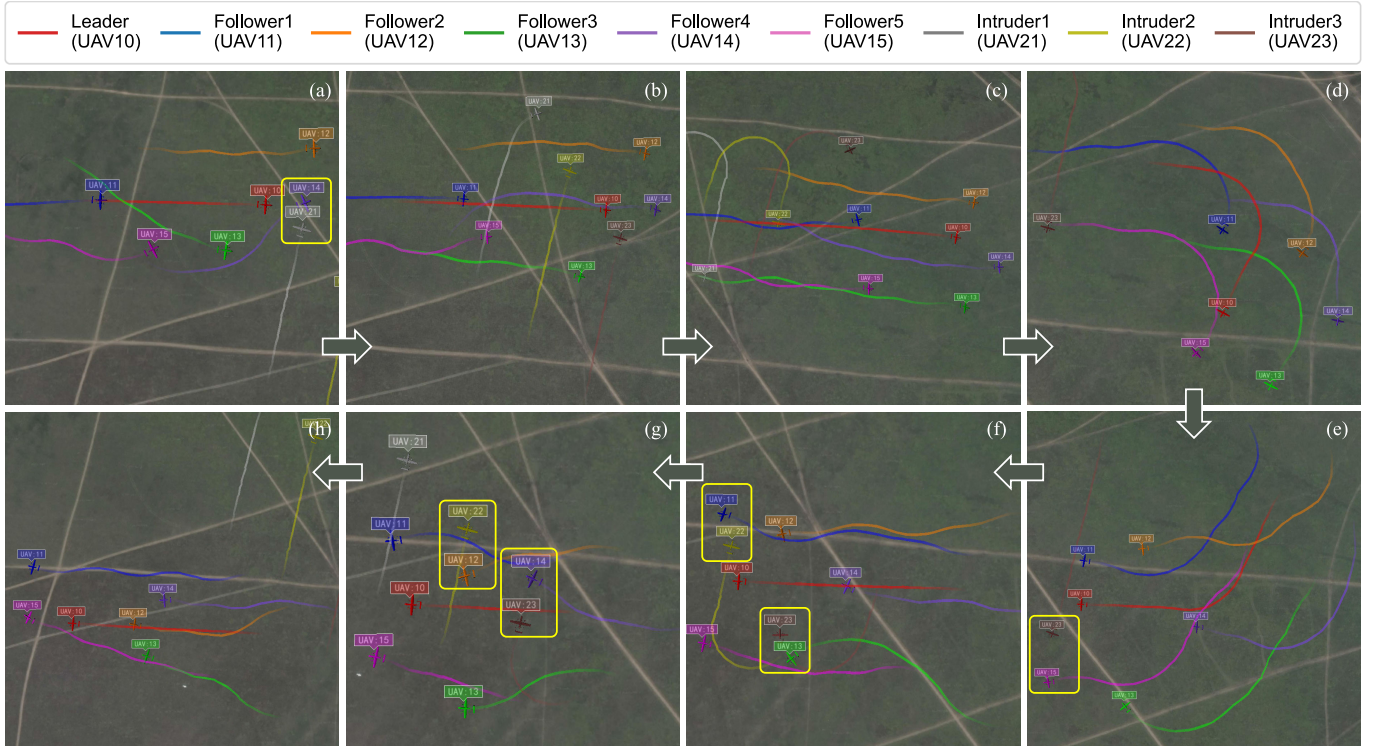


Fig. 8. Trajectories of the UAV fleet and three intruders in the HITL simulation. The eight snapshots are captured from the ground control station. (a) $t = 10$ s. (b) $t = 25$ s. (c) $t = 40$ s. (d) $t = 55$ s. (e) $t = 75$ s. (f) $t = 85$ s. (g) $t = 88$ s. (h) $t = 100$ s.

onboard computer to generate control actions using the learned STAAC policy is about 1.5 ms. This result indicates that the proposed algorithm can achieve real-time flocking control with limited computing capabilities.

We also recorded the performance of the learned STAAC policy in the HITL simulation. As shown in Figure 9(a), the distances from the leader to the followers (ρ) are kept within the range of 50 to 150 meters most of the time. This means that the followers can follow the leader steadily and avoid colliding

with the leader. The behavior of Follower1 (UAV11) is most striking. At the beginning, the distance between Follower1 and the leader is up to 232.74 m. After about 50 seconds, this distance decreases to less than 100 meters. Subsequently, this distance remains consistently between 50 and 100 meters. The above phenomenon indicates that the learned STAAC policy empowers the follower with the ability to form and maintain a cohesive flock. As shown in Figure 9(b) ~ (c), the minimum value of $D_{\min}^{\text{fol}}$ and $D_{\min}^{\text{int}}$ are both large than the minimum

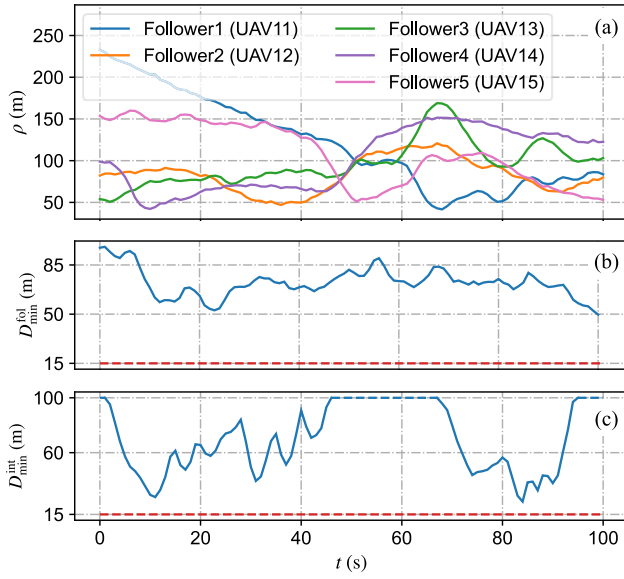


Fig. 9. Performance of the learned STAAC policy in the HITL simulation. (a) The distances to the leader. (b) The minimum distance among the followers. (c) The minimum distance between the followers and the intruders (Note that the upper limit is fixed at 100m for better visibility).

safety radius $R_s = 15$ m. These results demonstrate that using the learned STAAC policy, each follower is able to avoid collisions between other followers and the intruders at the same time. We can conclude that the flocking with collision avoidance policy learned from the numerical simulation environment using the proposed STAAC approach can be directly applied to the HITL simulation environment.

V. CONCLUSION

In this article, we have solved the distributed flocking with collision avoidance problem for a scalable fleet of fixed-wing UAVs by leveraging multi-agent reinforcement learning. Specifically, we have formulated this problem as a Dec-POMDP with a well-designed reward function. Then, we have proposed the STAAC algorithm to learn the distributed flocking with collision avoidance policy. In this algorithm, we have designed the spatial-temporal attention based population-invariant network architecture to improve the scalability. The results of numerical and HITL simulations have demonstrated that the flocking with collision avoidance policy learned with STAAC is able to enable the followers to flock with the leader without collision in dynamic environments with varying numbers of non-cooperative intruders. In the future, we will further investigate the flocking with collision avoidance problem for a fleet of fixed-wing UAVs in three-denominational environments with dynamic obstacles.

REFERENCES

- [1] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, "A survey on aerial swarm robotics," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 837–855, Aug. 2018.
- [2] J. Xie and J. Chen, "Multiregional coverage path planning for multiple energy constrained UAVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17366–17381, Oct. 2022.

- [3] Y. Wan, Y. Zhong, A. Ma, and L. Zhang, "An accurate UAV 3-D path planning method for disaster emergency response based on an improved multiobjective swarm intelligence algorithm," *IEEE Trans. Cybern.*, vol. 53, no. 4, pp. 2658–2671, Apr. 2023.
- [4] H. Liu, G. Wu, L. Zhou, W. Pedrycz, and P. N. Suganthan, "Tangent-based path planning for UAV in a 3-D low altitude urban environment," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12062–12077, Jun. 2023.
- [5] H. Duan, M. Huo, and Y. Fan, "From animal collective behaviors to swarm robotic cooperation," *Nat. Sci. Rev.*, vol. 10, no. 5, Apr. 2023, Art. no. nwad040.
- [6] B. D. Dalziel, M. Novak, J. R. Watson, and S. P. Ellner, "Collective behaviour can stabilize ecosystems," *Nature Ecology Evol.*, vol. 5, no. 10, pp. 1435–1440, Aug. 2021.
- [7] F. Wang, J. Huang, K. H. Low, and T. Hu, "Collective navigation of aerial vehicle swarms: A flocking inspired approach," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 1040–1053, Jun. 2024.
- [8] J. Qi, L. Bai, Y. Wei, H. Zhang, and Y. Xiao, "Emergence of adaptation of collective behavior based on visual perception," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10368–10384, Nov. 2023.
- [9] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, Sep. 2020.
- [10] W. Wang, L. Wang, J. Wu, X. Tao, and H. Wu, "Oracle-guided deep reinforcement learning for large-scale multi-UAVs flocking and navigation," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10280–10292, Oct. 2022.
- [11] J. Xiao, Z. Wang, J. He, and G. Yuan, "A graph neural network based deep reinforcement learning algorithm for multi-agent leader-follower flocking," *Inf. Sci.*, vol. 641, Sep. 2023, Art. no. 119074.
- [12] Y. Song et al., "Distributed swarm system with hybrid-flocking control for small fixed-wing UAVs: Algorithms and flight experiments," *Expert Syst. Appl.*, vol. 229, Nov. 2023, Art. no. 120457.
- [13] Z. Liu et al., "Mission-oriented miniature fixed-wing UAV swarms: A multilayered and distributed architecture," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 3, pp. 1588–1602, Mar. 2022.
- [14] S. Hung and S. N. Givigi, "A Q-learning approach to flocking with UAVs in a stochastic environment," *IEEE Trans. Cybern.*, vol. 47, no. 1, pp. 186–197, Jan. 2017.
- [15] C. Yan, C. Wang, X. Xiang, Z. Lan, and Y. Jiang, "Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing UAVs using local situation maps," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1260–1270, Feb. 2022.
- [16] C. Yan et al., "Pascal: Population-specific curriculum-based MADRL for collision-free flocking with large-scale fixed-wing UAV swarms," *Aerosp. Sci. Technol.*, vol. 133, Feb. 2023, Art. no. 108091.
- [17] S. Li, X. Cheng, X. Huang, S. A. Otaibi, and H. Wang, "Cooperative conflict detection and resolution and safety assessment for 6G enabled unmanned aerial vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2183–2198, Feb. 2023.
- [18] J. Ou, X. Guo, M. Zhu, and W. Lou, "Autonomous quadrotor obstacle avoidance based on dueling double deep recurrent Q-learning with monocular vision," *Neurocomputing*, vol. 441, pp. 300–310, Jun. 2021.
- [19] C. Wang, J. Wang, J. Wang, and X. Zhang, "Deep-reinforcement-learning-based autonomous UAV navigation with sparse rewards," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6180–6190, Jul. 2020.
- [20] G. Shen et al., "Deep reinforcement learning for flocking motion of multi-UAV systems: Learn from a digital twin," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11141–11153, Jul. 2022.
- [21] C. Bai, P. Yan, H. Piao, W. Pan, and J. Guo, "Learning-based multi-UAV flocking control with limited visual field and instinctive repulsion," *IEEE Trans. Cybern.*, vol. 54, no. 1, pp. 462–475, Oct. 2024.
- [22] C. Yan et al., "Collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments: A task-specific curriculum-based MADRL approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 10894–10908, Aug. 2024, doi: 10.1109/TNNLS.2023.3245124.
- [23] C. Yan, K. H. Low, X. Xiang, T. Hu, and L. Shen, "Attention-based population-invariant deep reinforcement learning for collision-free flocking with a scalable fixed-wing UAV swarm," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 13730–13736.
- [24] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, vol. 30, Dec. 2017, pp. 6382–6393.

- [25] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Int. Conf. Auto. Agents Multiagent Syst. (AAMAS)*, Sao Paulo, Brazil, 2017, pp. 66–83.
- [26] J. Ackermann, V. Gabler, T. Osa, and M. Sugiyama, "Reducing overestimation bias in multi-agent domains using double centralized critics," in *Proc. NeurIPS Workshop Deep RL*, 2019, pp. 1–10.
- [27] H. Chen, X. Wang, L. Shen, and Y. Cong, "Formation flight of fixed-wing UAV swarms: A group-based hierarchical approach," *Chin. J. Aeronaut.*, vol. 34, no. 2, pp. 504–515, Feb. 2021.
- [28] C. Yan, X. Xiang, and C. Wang, "Fixed-wing UAVs flocking in continuous spaces: A deep reinforcement learning approach," *Robot. Auto. Syst.*, vol. 131, Sep. 2020, Art. no. 103594.
- [29] H. Chen, Y. Cong, X. Wang, X. Xu, and L. Shen, "Coordinated path-following control of fixed-wing unmanned aerial vehicles," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 4, pp. 2540–2554, Apr. 2022.
- [30] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 107–118, Jan. 2021.
- [31] Z. Liu, Y. Cao, J. Chen, and J. Li, "A hierarchical reinforcement learning algorithm based on attention mechanism for UAV autonomous navigation," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 13309–13320, Nov. 2023.
- [32] H. Ryu, H. Shin, and J. Park, "Multi-agent actor-critic with hierarchical graph attention network," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 5, pp. 7236–7243.
- [33] T. Ni, B. Eysenbach, and R. Salakhutdinov, "Recurrent model-free RL can be a strong baseline for many POMDPs," in *Proc. Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 16691–16723.
- [34] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [35] C. Bai, P. Yan, W. Pan, and J. Guo, "Learning-based multi-robot formation control with obstacle avoidance," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11811–11822, Aug. 2022.
- [36] Z. Zhang, D. Zhang, Q. Zhang, W. Pan, and T. Hu, "DACOOP-A: Decentralized adaptive cooperative pursuit via attention," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5504–5511, Jun. 2024, doi: [10.1109/LRA.2023.3331886](https://doi.org/10.1109/LRA.2023.3331886).
- [37] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal N-body collision avoidance," in *Proc. Int. Symp. Robotic Res.* Berlin, Germany: Springer, 2011, pp. 3–19.



learning, developmental robotics, and human–robot teamwork.



Chang Wang received the B.S. degree in mathematics from the University of Science and Technology of China, Hefei, China, in 2003, the M.Sc. degree in applied mathematics from the National University of Defense Technology (NUDT), Changsha, China, in 2009, and the Ph.D. degree in robot learning from Delft University of Technology, Delft, The Netherlands, in 2016.

He is currently an Associate Professor with the College of Intelligence Science and Technology, NUDT. His research interests include reinforcement

Han Zhou received the B.E., M.S., and Ph.D. degrees in control science and engineering from the National University of Defense Technology (NUDT), Changsha, China, in 2007, 2009, and 2015, respectively.

She is currently an Associate Professor with the College of Intelligence Science and Technology, NUDT. Her current research interests include the cooperative control of unmanned systems and biomimetic robotics.



Xiaojia Xiang received the B.E., M.S., and Ph.D. degrees in automatic control from the National University of Defense Technology (NUDT), Changsha, China, in 2003, 2007, and 2016, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. His research interests include mission planning, autonomous, and cooperative control of unmanned systems.



Xiangke Wang (Senior Member, IEEE) received the B.E., M.E., and Ph.D. degrees in control science and engineering from the National University of Defense Technology (NUDT), Changsha, China, in 2004, 2006, and 2012, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. He has authored or co-authored three books and more than 100 publications in peer-reviewed journals and international conferences. His current research interests include the control of multi-agent systems and their

applications on unmanned aerial vehicles.



Chao Yan (Member, IEEE) received the B.E. degree in electrical engineering and automation from China University of Mining and Technology, Xuzhou, China, in 2017, and the M.S. and Ph.D. degrees in control science and engineering from the National University of Defense Technology, Changsha, China, in 2019 and 2023, respectively.

He was a Visiting Ph.D. Student with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore, from 2021 to 2022. He is currently an Associate

Professor with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His research interests include deep reinforcement learning and coordination control of UAV swarms.



Lincheng Shen received the B.E., M.E., and Ph.D. degrees in automatic control from the National University of Defense Technology, Changsha, China, in 1986, 1989, and 1994, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, National University of Defense Technology, where he serves as the Dean of the Graduate School. His research interests include unmanned aerial vehicles, swarm robotics, and artificial intelligence.

Dr. Shen has initiated and organized several workshops and symposia, including the International Workshop on Bionic Engineering in 2012 and Chinese Automation Congress in 2013. He has been serving as an Editorial Board Member of *Journal of Bionic Engineering* since 2007.